

Computación Ubicua

*PECL4 – Documentación del proyecto de
laboratorio*

Universidad de Alcalá



Sergio Bardavio Soria
Ángel Jiménez Moneva

Febrero 2025-2026

Índice

Índice	2
1. Introducción.....	5
2. Objetivos y alcance del proyecto.....	5
3. Proyectos de referencia.....	6
3.1. Referentes Comerciales e Industriales.....	6
3.2. Referentes de la Comunidad (DIY / Open Source).....	7
3.3. Análisis Comparativo y Posicionamiento	8
4. Desarrollo	8
4.1. Arquitectura del sistema	8
4.2. Creación de la infraestructura	11
4.3. Capa de transporte.....	13
4.4. Capa de percepción	14
4.4.1. Sensores/Actuadores	14
4.4.2. Construcción del prototipo	15
4.5. Comunicación con otras capas.....	18
4.6. Pruebas realizadas	19
4.7. Capa de procesado	20
4.7.1. Base de datos.....	21
4.7.2. Desarrollo del proyecto servidor	22
4.8. Capa de aplicación	23
4.8.1. Aplicación móvil.....	24
4.8.2. Aplicación Web.....	28
4.9. Planificación del proyecto.....	30
5. Conclusiones.....	30
6. Bibliografía.....	32
Anexo I – Manual de instalación.....	33
Anexo II – Manual de usuario de la aplicación móvil.....	34
Anexo III – Manual de usuario de la aplicación web	38
Anexo IV – Hojas de características	42
1. Microcontrolador: DOIT ESP32 DevKit V1.....	42
2. Sensor de humedad y temperatura: DHT11	43

3. Sensor de iluminación: GL5528	44
4. Pantalla LCD 20x4 con interfaz I2C	45

Índice de Figuras

Ilustración 1 - Arquitectura del sistema.....	9
Ilustración 2 - Prototipo.....	16
Ilustración 3 - Montaje real	18
Ilustración 4 - Modelo relacional	21
Ilustración 5 - Diagrama de tablas.....	22
Ilustración 6 - Pantallas aplicación móvil	25
Ilustración 7 - Pantallas aplicación web.....	29
Ilustración 8 - Microcontrolador ESP32	43
Ilustración 9 - Sensor de humedad y temperatura DHT11	44
Ilustración 10 - Sensor de iluminación GL5528	45
Ilustración 11- Pantalla LCD 20x4 con interfaz I2C.....	46

1. Introducción

En este documento se detalla la memoria final del proyecto que hemos desarrollado, el cuál que consiste en obtener información sobre distintas pantallas de información repartidas entre la ciudad y poder acceder a esta a través de una aplicación para Android. Para desarrollarla hemos tenido en cuenta el contexto global donde las Smart Cities buscan optimizar la gestión de recursos y mejorar la calidad de vida de los ciudadanos, por lo que surge la necesidad de implementar sistemas de monitorización ambiental robustos y escalables.

En este proyecto hemos llevado a cabo el diseño e implementación de un sistema integral de gestión urbana denominado AtmoCity. El proyecto abarca desde el despliegue de nodos sensores físicos hasta la creación de una interfaz de usuario móvil, pasando por una infraestructura de red basada en contenedores y servicios web. A través de este documento, el lector podrá comprender cómo se integra el hardware de bajo coste con protocolos de comunicación industriales para crear una solución de ciudad inteligente funcional.

La memoria tiene la siguiente estructura: se divide en cinco bloques principales, después de esta introducción y la definición de objetivos, se explica la información sobre los proyectos de referencia. Tras esto estará el desarrollo técnico del proyecto, donde se detallan las capas de infraestructura (Docker), transporte (MQTT), percepción (ESP32 y sensores), procesamiento (Servidor Tomcat y MariaDB) y aplicación (Android). Finalmente, se incluye la planificación temporal del proyecto, las conclusiones extraídas y los manuales de uso obligatorios.

2. Objetivos y alcance del proyecto

Este proyecto nace de la necesidad de modernizar el mobiliario urbano. Hemos detectado que los paneles informativos tradicionales son elementos estáticos que no aportan datos en tiempo real. Por ello el objetivo principal de este trabajo ha sido diseñar y construir un prototipo funcional de Pantalla de Información Inteligente, transformando un simple cartel en un nodo IoT completo capaz de percibir su entorno y comunicarse.

El funcionamiento del sistema que hemos desarrollado es muy específico y abarca todo el ciclo de vida del dato. En la capa de captura, hemos creado una estación basada en un microcontrolador ESP32 que recoge métricas ambientales cada 10 segundos utilizando sensores DHT11 (para temperatura y humedad) y sensores de luz. Esta información no se queda en la placa, sino que se empaqueta automáticamente en formato JSON y se transmite mediante el protocolo MQTT a un bróker centralizado, garantizando así una comunicación ligera y rápida.

Para ser capaces de gestionar toda esta información, el alcance del proyecto incluye una infraestructura de servidor robusta. Hemos configurado un servidor intermedio Tomcat

que se suscribe a los datos que llegan por MQTT, los procesa y los almacena de forma persistente en una base de datos relacional MariaDB mediante un pool de conexiones. A su vez, este servidor expone una API REST con varios endpoints que permitirá que la información histórica esté disponible para ser consultada de forma externa y estandarizada. Además, para mejorar el manejo de los datos en la base de datos, hemos implementado una aplicación web Adminer que permite realizar todo tipo de consultas sobre las tablas de la base de datos, así como ver los distintos registros guardados en estas.

La interacción final con el usuario se realiza a través de una aplicación Android que hemos diseñado, esta utiliza la API REST para consultar los datos de las distintas calles, permitiendo aplicar filtros para la fecha, temperatura y humedad, ofreciendo así la capacidad de ver los historiales completos de las distintas calles y gráficos de temperatura y humedad de estas. También se conecta al bróker MQTT para visualizar qué está pasando exactamente ahora mismo en la calle, siendo capaz de mostrar los datos en tiempo real de las distintas calles, así como su ubicación en el mapa basándose en la longitud y latitud (para ello hemos usado una API de Google Maps).

Por último, cabe destacar que el sistema no es solo de lectura, sino que hemos dotado a la aplicación de la funcionalidad para enviar mensajes hacia la pantalla LCD del dispositivo físico (que imprimirá al instante). Esto permite a un operario mostrar alertas de tráfico o mensajes personalizados en la calle de forma instantánea desde su móvil, cerrando así el círculo de comunicación bidireccional entre el ciudadano y los gestores municipales. Para realizarlo usamos el API REST de la aplicación de Android para llamar al endpoint SendMessage, que crea una conexión MQTT y envía un mensaje al topic que estamos suscritos en el ESP32, una vez este detecta que ha recibido el mensaje lo imprime por pantalla.

3. Proyectos de referencia

Antes de comenzar el desarrollo técnico de nuestro proyecto, hemos realizado un análisis para entender qué tecnologías se están utilizando actualmente en el mundo real. Al tratarse de un proyecto de Smart Cities centrado en Pantallas de Información, hemos encontrado dos grandes grupos de referentes, estos son los referentes industriales que cuentan con soluciones complejas y caras, y los referentes Maker (DIY)

En este apartado analizaremos los dos grupos referentes y explicaremos las características que comparte nuestro proyecto con estos, así como las diferencias.

3.1. Referentes Comerciales e Industriales

En el mundo actual, las Smart Cities mueven millones de euros y ya existen empresas que ya despliegan infraestructuras masivas.

El primer referente que hemos analizado es la empresa española Libelium, que es líder mundial en IoT. Ellos comercializan la plataforma Waspmote Smart Environment, esta cuenta con unos nodos de sensores muy robustos (encapsulados para resistir lluvia y golpes) que miden la calidad del aire, ruido y temperatura en las ciudades.

Este referente se relaciona con nuestro proyecto en nuestra Capa de Percepción (PL1) donde obtenemos la información de los sensores y la mostramos por pantalla. Aunque nosotros utilizamos un ESP32 y sensores económicos como el DHT11, la lógica que empleamos es la misma, donde un dispositivo autónomo lee el entorno y envía los datos a la nube. La diferencia es que Libelium usa protocolos industriales muy caros, mientras que nosotros hemos optado por MQTT sobre WiFi, que es mucho más accesible y ligero para nuestro prototipo.

El segundo referente comercial son los Soportes Digitales Informativos (Mupis) que empresas como JCDecaux o Clear Channel instalan en paradas de autobús y plazas. Son esas grandes pantallas verticales que muestran publicidad, la hora y la temperatura.

Nuestro proyecto sería una mejora de este referente ya que los Mupis comerciales suelen ser simples dado a que solo muestran lo que les mandan desde una central (publicidad) y no suelen interactuar con el entorno. Nuestro proyecto incorpora sensores a la pantalla, logrando así que el dispositivo no solo muestre la información que le envían desde la aplicación (bidireccionalidad) sino que también obtenga la información sobre temperatura, humedad y luz y la muestre. Nuestra App Android, nos permite enviar mensajes en tiempo real a la pantalla que es algo que en los sistemas comerciales suele ser un proceso burocrático lento.

3.2. Referentes de la Comunidad (DIY / Open Source)

Existen también proyectos de gran calidad creados por programadores independientes que comparten su código, y que se parecen mucho a la filosofía de nuestra práctica.

El proyecto más famoso es el MagicMirror, que se trata de un sistema modular donde la gente utiliza una Raspberry Pi conectada a un monitor (detrás de un espejo) para visualizar su agenda, el tiempo, las noticias RSS o el estado del tráfico antes de salir de casa.

Esto se relaciona con nuestro proyecto en la visualización de datos. Pero cuenta con diferencias en la arquitectura, dado a que MagicMirror está pensado para funcionar de forma aislada, mientras que nuestro proyecto ha sido diseñado desde el principio con una arquitectura de microservicios en Docker (PL2). Esto significa que nuestro sistema no está pensado para una sola pantalla, sino para gestionar cientos de ellas de forma centralizada, por lo que si quisiéramos crear 100 pantallas y repartirlas por Madrid, nuestro proyecto sería mucho más eficiente que MagicMirror.

También hemos tomado como referencia la plataforma OpenSenseMap (asociada al proyecto senseBox). Es una iniciativa donde ciudadanos de todo el mundo montan sus estaciones meteorológicas con Arduino y suben los datos a un mapa web global.

Esto se relaciona con nuestro proyecto ya que es la misma idea de lo que hacemos en nuestra Capa de Aplicación (PL3). Al igual que OpenSenseMap, nuestra App utiliza Google Maps para geolocalizar los dispositivos, además nuestro trabajo tiene una mejora respecto a este referente, que es la capacidad de actuación ya que en OpenSenseMap solo puedes mirar datos, mientras que desde la aplicación de Android puedes enviar mensajes a las distintas pantallas e interactuar con ellas.

3.3. Análisis Comparativo y Posicionamiento

Después de llevar a cabo el análisis de los distintos tipos de referentes podemos concluir que nuestro proyecto se sitúa en un punto intermedio híbrido, donde contamos con:

- **Hardware Accesible:** Utilizamos componentes económicos (ESP32) propios del mundo DIY, lo que haría viable llenar una ciudad de estos dispositivos sin gastar el presupuesto de Libelium.
- **Software Profesional:** A diferencia de los proyectos caseros, hemos implementado una infraestructura de servidor "Enterprise" (Docker, Base de Datos Relacional, API REST y Pool de Conexiones).

En resumen, nuestra solución extiende el uso de la tecnología de las Smart Cities ofreciendo la potencia de gestión de los sistemas industriales, pero con la flexibilidad y el coste reducido de los proyectos de comunidad.

4. Desarrollo

En este apartado se aportará todo el estudio realizado para abordar el desarrollo del proyecto. El contenido se puede dividir en las siguientes subsecciones:

4.1. Arquitectura del sistema

Para desarrollar el proyecto hemos tenido que abordar la complejidad de un sistema de Computación Ubicua que integra elementos diferentes como sensores físicos, bases de datos y dispositivos móviles. Hemos diseñado una arquitectura modular distribuida en tres capas claramente diferenciadas. Esta estructura sigue el modelo de referencia estándar para sistemas IoT (Internet of Things), garantizando que cada componente tenga una responsabilidad única y que el sistema sea escalable.

A nivel global, el sistema cuenta con una arquitectura híbrida. Por un lado, utiliza un modelo Publicador/Suscriptor (basado en el protocolo MQTT) para la transmisión de

datos en tiempo real y la comunicación asíncrona, logrando así pasar los datos del ESP32 a la aplicación de Android. Por otro lado, implementa un modelo clásico Cliente-Servidor (basado en HTTP/REST) para la gestión de datos persistentes, pudiendo así acceder a los registros de datos de cada calle en Android.

A continuación, se presenta el esquema visual de la arquitectura propuesta y la descripción detallada de sus componentes:

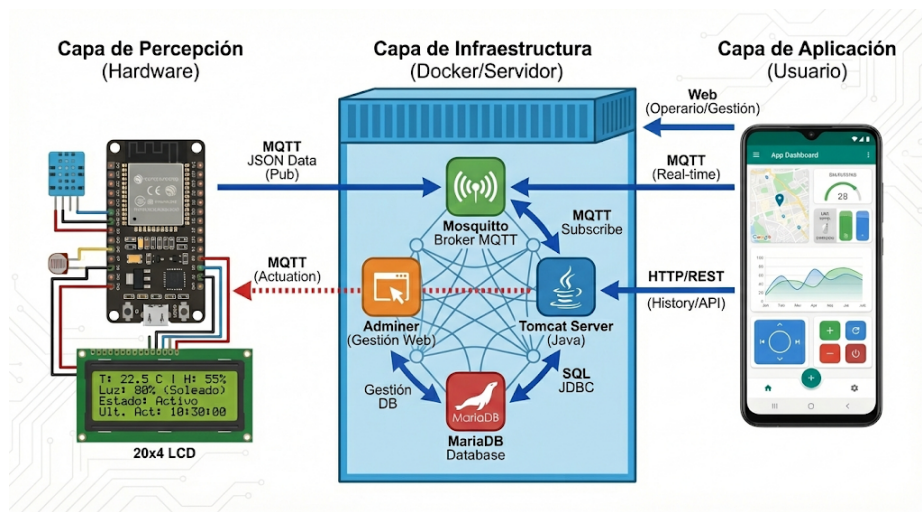


Ilustración 1 - Arquitectura del sistema

Descripción de los componentes y evolución del despliegue

Capa de Percepción (Hardware y Firmware Evolutivo)

En el extremo izquierdo se encuentra la capa de percepción (hardware) que está basado en un microcontrolador ESP32 con sensores DHT11 y de luminosidad. De esta capa cabe destacar la evolución que ha tenido a lo largo del proyecto (entrega de las distintas PL).

Originalmente, el sistema se diseñó con un despliegue estático donde el hardware enviaba datos de un único identificador de sensor (ST_0155) hacia el broker, esto sería así para las dos primeras entregas.

Para validar la robustez de la aplicación Android y su capacidad de geolocalización, optimizamos el firmware del dispositivo implementando una simulación de despliegue multi-calle. Por lo que cambiamos el programa de Arduino para que un solo microcontrolador físico (nuestro ESP32) sea capaz de generar tráfico de datos para tres ubicaciones distintas (Calle de los Montes Albares, Calle Pedro Gumiél y Paseo del Prado), variando ligeramente los valores biométricos en cada envío, teniendo así cada

envío su propio topic (cambiamos el `street_ID`) y json. Esto nos ha permitido comprobar cómo la Capa de Aplicación discrimina y filtra los datos por zonas geográficas sin necesidad de adquirir tres placas físicas. Además de asegurarnos que la aplicación funcionará en caso de que hubiese varias pantallas mandando información.

Capa de Infraestructura (Middleware y Gestión)

En el centro tenemos la capa de infraestructura (Middleware y Gestión), para esta hemos virtualizado toda la lógica de negocio utilizando Docker. Dentro de esta infraestructura conviven cuatro servicios clave interconectados mediante una red interna y todos se levantarán a la vez gracias al docker compose:

- ❑ **Broker MQTT (Mosquitto):** Este servidor actúa como el núcleo de comunicaciones en tiempo real, operando en el puerto 1883. Su función consiste en desacoplar el hardware del software, recibiendo los mensajes JSON de los sensores y distribuyéndolos instantáneamente a los suscriptores activos (Servidor y App) de forma ligera y asíncrona.
- ❑ **Servidor de Aplicaciones (Tomcat):** Este servidor aloja la lógica de negocio desarrollada en Java y actúa como puente inteligente entre los protocolos. Gestiona la suscripción a los tópicos MQTT para capturar los datos en vivo y meter los datos en la base de datos de MariaDB mediante un pool de conexiones, además expone una API REST segura con los distintos endpoints para que la aplicación móvil pueda realizar consultas complejas sobre el historial.
- ❑ **Base de Datos (MariaDB):** Este servidor proporciona la capa de persistencia relacional del sistema, garantizando así la integridad de la información a largo plazo. Nuestra base de datos cuenta con 3 tablas, estas son Calle, Dispositivo y Telemetría. Esto nos permite garantizar la integridad de los datos y evitar las redundancias entre ellos, además de facilitar la obtención de endpoints necesarios para la aplicación. Almacena el histórico de las distintas tablas que nos permitirá tener persistencia en todo el proyecto, que será necesario para la generación posterior de estadísticas y gráficas temporales en la aplicación.
- ❑ **Servicio de Gestión (Adminer):** Este servicio es una herramienta (aplicación) web ligera conectada directamente a la base de datos. Y será vital para la arquitectura, ya que proporciona a los administradores una interfaz visual para realizar tareas de mantenimiento, verificación de datos y todo tipo de consultas sobre los datos sin depender de la consola de comandos.

Capa de Aplicación (Usuario Final)

Finalmente, la Aplicación Android cierra el ciclo interactuando con las capas anteriores de forma simultánea. Esta está suscrita al Broker MQTT para poder obtener la

información en tiempo real de las distintas pantallas de información (simulación multicalle explicada en la capa de percepción). Además, consume la API REST del servidor Tomcat para ver el historial de datos de cada una de las calles, teniendo la opción de aplicarle filtros a estos registros, así como obtener gráficos de cada calle.

Flujos de información

El sistema se caracteriza por ser bidireccional, por lo que la aplicación de Android puede escribir un mensaje que se mostrará en la pantalla de información, para ello cuenta con:

Flujo Ascendente (Monitorización) donde los datos simulados de las tres calles viajan desde el ESP32 por el Broker MQTT, a los topic que envía el ESP32 (Sensors/street_id/sensor/telemetry) está suscrito tanto el servidor de tomcat que procesará los datos y los enviará al MariaDB, para así obtener el historial de estos en la aplicación de Android usando el API REST, como la aplicación donde una vez seleccionas una calle podrás ver los datos en tiempo real de estos. Al suscribirnos al topic de MQTT donde está el street_id tenemos un + (wildcard), indicando así que podríamos recibir datos de cualquier calle y funcionará todo correctamente.

Flujo Descendente (Actuación) donde las órdenes viajan desde la App Android hacia el servidor tomcat, aquí usará el endpoint de sendMessage que crea una conexión MQTT y publica en el topic Sensors/street_id/actuador/mensaje, al cual está suscrito el ESP32, que cuando recibe un mensaje lo procesa y lo imprime por pantalla.

4.2. Creación de la infraestructura

Para el desarrollo de la infraestructura de soporte del proyecto, hemos tomado la decisión de utilizar Docker como motor de virtualización. Esto nos permite crear un entorno de ejecución robusto, estandarizado y portátil, eliminando los problemas de compatibilidad entre diferentes ordenadores y facilitando el despliegue sin necesidad de instalaciones complejas en el sistema operativo anfitrión.

Toda la definición de la infraestructura se encuentra orquestada mediante Docker Compose, centralizando la configuración del archivo docker-compose.yml. Los aspectos técnicos más relevantes de esta implementación son:

El uso de una Red Privada, para ello hemos definido una red puente personalizada (app-network) que permite que los contenedores se comuniquen entre sí por su nombre de servicio (mariadb-server, tomcat-server) sin exponer sus puertos internos innecesariamente.

Gestión de credenciales, para evitar exponer credenciales sensibles en el código fuente, hemos implementado el uso de variables de entorno. Las contraseñas (como la de la base de datos) y los puertos externos se almacenan en un fichero oculto .env, que Docker lee en tiempo de ejecución para configurar los servicios de forma segura.

La infraestructura se compone de cuatro servicios fundamentales definidos en el fichero del docker compose, estos son:

El servicio de Broker MQTT (mqtt-broker), para el cuál hemos utilizado la imagen oficial eclipse-mosquitto:latest. Este contenedor expone el puerto 1883 al host para permitir la conexión de los dispositivos físicos (ESP32) y la aplicación móvil. Además, hemos configurado volúmenes para persistir la configuración y los logs del servicio.

El servicio de Base de Datos (mariadb-server) que utiliza la imagen oficial de MariaDB. Este servicio es necesario para guardar la persistencia sobre los datos de las distintas calles, para ello empleamos bind mounts que asegura que los datos físicos de la ciudad se guarden en una carpeta local del proyecto, garantizando así la portabilidad total, permitiendo mover el proyecto a otro ordenador sin perder el histórico de registros. Además, hemos utilizado el mecanismo de entryptpoint para inyectar un script SQL de inicialización (init.sql) al arrancar el contenedor, lo que permite que la base de datos cree automáticamente toda la estructura de tablas y relaciones la primera vez que se despliega, sin necesidad de ejecutar comandos SQL manualmente.

Por seguridad, no expone puertos al exterior, aunque escucha internamente en el puerto 3306, siendo accesible únicamente por el servidor Tomcat y el gestor Adminer dentro de la red Docker. Para configurar este servidor, inyectamos variables de entorno (como MARIADB_ROOT_PASSWORD y MARIADB_DATABASE) que se leen desde un fichero .env oculto, garantizando así que las credenciales no estén expuestas en el código fuente.

El servidor de Aplicaciones (tomcat-server), este a diferencia de los anteriores se construye a partir de una imagen personalizada (Dockerfile multi-etapa). Para el buen funcionamiento de este hemos definido una dependencia estricta (depends_on: service_healthy) con respecto a MariaDB y Mosquitto, asegurando que el servidor Tomcat no arranque hasta que la base de datos y el bróker estén totalmente operativos. Este servidor expone el puerto 8080 para las peticiones API REST.

En este dockerfile usamos una imagen Maven para compilar el código fuente Java y generar el archivo .war, y despliega dicho .war sobre una imagen Tomcat 11 (JDK 17) limpia. Esto lo hacemos para que se pueda ejecutar desde cualquier dispositivo sin necesidad de tener java descargado.

El servidor Gestor de Base de Datos (adminer), que ha sido creado como herramienta de soporte exponiendo el puerto 8081. Al estar dentro de la misma app-network, este servicio se conecta directamente a MariaDB sin necesidad de exponer el puerto de la base de datos a internet, aumentando la seguridad

La infraestructura desplegada constituye una base sólida y profesional para el proyecto. Gracias al uso de Docker y la construcción personalizada de imágenes, hemos logrado un sistema autocontenido (no requiere instalar Java ni MySQL en el PC), seguro (gracias a la segregación de redes y variables de entorno) y altamente escalable, capaz de soportar la carga de trabajo de una Smart City simulada.

4.3. Capa de transporte

La capa de transporte es necesaria para realizar las comunicaciones del proyecto, esta es la encargada de orquestar el flujo de información entre el hardware, el servidor de procesado y la interfaz de usuario móvil. Para implementar esta capa hemos descartado el uso de protocolos web tradicionales como HTTP para la transmisión en tiempo real, ya que su modelo de petición-respuesta genera una sobrecarga excesiva, lo que provocaría un mal funcionamiento de la aplicación, especialmente para los datos en tiempo real. En su lugar, hemos optado por MQTT (Message Queuing Telemetry Transport), que es el estándar de referencia en la industria IoT, ya que cuenta con una arquitectura ligera orientada a eventos, un bajo consumo de ancho de banda y una gran capacidad para operar de forma fiable incluso en redes con conectividad inestable.

El servicio que gestiona este tráfico es el Broker MQTT, implementado mediante Eclipse Mosquitto y desplegado en el puerto 1883 de nuestra infraestructura Docker. La elección de usar la imagen de Mosquitto nos aporta una solución extremadamente eficiente que, a diferencia de otros gestores de mensajería más pesados, consume recursos mínimos de CPU y memoria. Su función principal es actuar como intermediario asíncrono, permitiendo un desacoplamiento total entre el ESP32 (emisor) y la App (receptor), y permitiendo que estos nunca se comunican directamente. Esto garantiza que el sistema sea robusto ante fallos, dado a que si la aplicación móvil pierde cobertura, el hardware puede seguir enviando datos sin bloquearse, gestionando el broker la entrega de mensajes gracias a los niveles de Calidad de Servicio (QoS) integrados en el protocolo.

La organización lógica de la comunicación se estructura mediante un sistema jerárquico de canales. Para realizar la simulación de múltiples ubicaciones a partir de un único ESP32 y garantizar la escalabilidad, contamos con esta estructura para los topics de las distintas calles (Sensors/street_id/sensor/telemetry), donde el street_id será el de cada una de las calles (ST_0155, ST_0156, ST_0157).

En el flujo ascendente de información, el microcontrolador ESP32 publica periódicamente los datos ambientales variando el identificador de la calle (street_id) en cada publish, por lo que cada 10 segundos hace 3 publish distintos, uno de cada una de las calles. Esta simulación nos permite comprobar el buen funcionamiento del servidor Tomcat y la aplicación de Android, siendo capaces de obtener los datos de todas las pantallas de información que quisiéramos incorporar. Esto lo consiguen usando el mecanismo de suscripción mediante comodines (Wildcards). En lugar de suscribirse individualmente a cada calle, los clientes escuchan el tópico Sensors/+sensor/telemetry, permitiendo recibir automáticamente los mensajes de cualquier calle existente mediante un único canal de entrada.

En el flujo descendente usaremos wildcards, ya que a pesar de realizar con nuestro ESP32 el simulamiento de tres calles, en realidad solo contamos con una pantalla de información, por lo que los mensajes que queramos enviar desde la aplicación de cualquiera de las tres calles nos llegarán a la única pantalla real que tenemos. Cuando desde la aplicación quieras imprimir un mensaje por la pantalla de información, llamas

al API REST para usar el endpoint de sendMessage, este endpoint hace publish del mensaje al topic dependiendo del street_id, y al usar wildcard nuestro único ESP32 esta suscrito a todos los mensajes de las distintas calles y los publica por pantalla.

Para el desarrollo del proyecto, la interconexión se logra estableciendo canales de comunicación persistentes contra el Broker central. El dispositivo físico (ESP32) inicia la cadena actuando como emisor, haciendo los publish cada 10 segundos. Simultáneamente, tanto el backend en el servidor como la aplicación móvil mantienen una escucha usando los listeners sobre estos mismos canales, reaccionando instantáneamente a la llegada de nuevos paquetes. El contenido de estos mensajes se ha estandarizado utilizando el formato JSON, por lo que cada publicación encapsula una estructura de texto ligera con los pares clave-valor de las lecturas, garantizando así la interoperabilidad, permitiendo que el servidor procese los datos para persistirlos y que la App los interprete para actualizar la interfaz gráfica.

4.4. Capa de percepción

En este apartado se analizarán los elementos que componen la capa de percepción, indicando primero el objetivo de esta capa y después descomponiendo el resto en subapartados que traten sobre cada uno de los elementos de la capa.

4.4.1. Sensores/Actuadores

La capa de percepción constituye la frontera física del proyecto *AtmoCity*, siendo el punto de contacto directo con el entorno urbano. Su función principal es la recolección de datos, siendo responsable de digitalizar el estado ambiental de la ciudad (temperatura, humedad y luz) para alimentar la toma de decisiones en la nube y, simultáneamente, actuar como un punto de información visual para el ciudadano. A continuación, se detalla la contribución técnica de cada uno de los distintos componentes del sistema:

- **Microcontrolador ESP32 DevKit V1:** Este dispositivo actúa como el cerebro del Edge Node, proporcionando la potencia de procesamiento necesaria para gestionar la pila de protocolos WiFi y MQTT. Su función consiste en ejecutar la lógica de simulación multi-calle programada en el firmware, tomando los datos base y aplicando algoritmos de variación para generar tráfico diferenciado para tres ubicaciones (Montes Albares, Pedro Gumiel y Paseo del Prado), validando así la escalabilidad del sistema con un único hardware.
- **Sensor de Temperatura y Humedad (DHT11):** Conectado físicamente al pin digital 17, este sensor es el encargado de monitorizar el confort térmico y la humedad relativa del entorno. El firmware utiliza la medición real de la habitación como base matemática para calcular las fluctuaciones de las calles

virtuales, garantizando que los datos simulados que llegan a la aplicación sean coherentes y verosímiles.

- **Sensor de Luz Digital (LDR):** Ubicado en el pin digital 25 y configurado con resistencia interna (INPUT_PULLUP), este componente detecta la luminosidad ambiental y la traduce en una señal digital simplificada de brillo, el valor será 0 o 100 dependiendo si la luz esta encendida o apagada. Este componente sería esencial en una smart city para situaciones como saber si es de día o de noche para activar o desactivar virtualmente el sistema de alumbrado público y ahorrar energía.
- **Pantalla LCD 20x4 (Actuador):** Integrada mediante el bus I2C en los pines 21 (SDA) y 22 (SCL), constituye la interfaz humano-máquina (HMI) del sistema. Funciona como un actuador visual bidireccional del sistema, donde escribe cada 10 segundos las distintas mediciones locales, y reacciona a los comandos remotos enviados desde la App Android imprimiéndolos por pantalla. Esto permite a los operarios proyectar mensajes de texto de emergencia y activar parpadeos de la retroiluminación para alertar a los ciudadanos ante incidencias críticas.

4.4.2. Construcción del prototipo

Para la materialización física del sistema AtmoCity, hemos decidido usar un diseño modular implementado sobre una placa de pruebas. Esta decisión facilita la iteración rápida y el mantenimiento del hardware. El proceso de construcción se divide en dos fases principales, estas son el interconexionado físico de los componentes y la implementación de la lógica de control.

Montaje Físico y Conexiones

El núcleo del sistema es la placa ESP32 DevKit V1. La estrategia de cableado se ha diseñado para optimizar el uso de los pines del microcontrolador, reservando las entradas analógicas y digitales directas únicamente para los sensores que lo requieren y utilizando buses de comunicación para los periféricos más complejos. Todo el sistema comparte un bus de alimentación común, tomando la energía directamente de los pines de 5V (VIN) y GND del ESP32, esto que garantiza la tensión necesaria para el funcionamiento estable tanto del sensor DHT11 como de la retroiluminación de la pantalla.

Respecto a la interfaz de visualización, para evitar la ocupación masiva de pines digitales que requeriría una conexión paralela, hemos conectado la pantalla LCD 20x4 mediante el protocolo I2C, por lo que se reduce el cableado a solo dos líneas de datos conectadas a los pines GPIO 21 (SDA) y GPIO 22 (SCL). Por otro lado, la captura de

los datos ambientales se realiza a través del sensor DHT11 conectado al pin GPIO 17, habiéndose verificado la estabilidad eléctrica de la señal para obtener lecturas digitales fiables. Para la detección de luz el sensor LDR se ha conectado al pin GPIO 25, que cuenta con el uso de la resistencia de pull-up interna del propio ESP32 y al configurarla mediante software (INPUT_PULLUP), hemos logrado eliminar componentes pasivos externos, simplificando el circuito físico y permitiendo detectar la luminosidad como cambios de estado lógico.

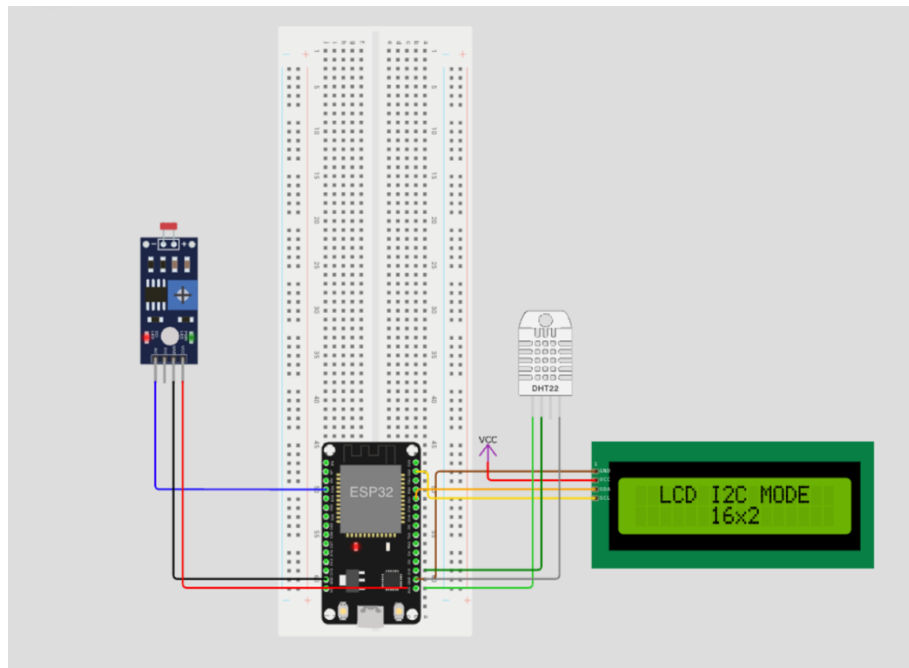


Ilustración 2 - Prototipo

Programación del Dispositivo (Firmware)

La lógica de control que dota de inteligencia al nodo ha sido utilizando el IDE de Arduino, en esta el firmware integra librerías estándar para la gestión de la pila TCP/IP y, destacando la librería PubSubClient para la comunicación MQTT. El flujo del programa comienza con una fase de `setUp`, donde después de inicializar los sensores, el dispositivo establece conexión con el punto de acceso WiFi ("POCO X7 Pro"). Además en esta etapa ampliamos el tamaño del buffer de transmisión a 4096 bytes, para así permitir el envío de cadenas JSON extensas sin que se produjera fragmentación o pérdida de datos.

Luego llegaremos al loop que ejecuta un ciclo de telemetría cada 10 segundos. Es aquí donde se implementa la lógica de simulación multi-calle. En lugar de realizar una lectura y un envío simple de una calle, para la última entrega hicimos que el firmware tome los datos reales y entre en un proceso iterativo que recorre una estructura de datos personalizada llamada SensorConfig. Esta estructura contiene los identificadores de las tres calles virtuales: "Montes Albères", "Pedro Gumiel" y "Paseo del Prado".

Para cada una de estas iteraciones, el código aplica un algoritmo de variación mediante offsets aleatorios sobre los datos base. Esto permite que, a partir de un único sensor físico, se generen tres calles virtuales independientes con valores ligeramente distintos, que son empaquetados en objetos JSON y publicados en sus respectivos topics. Simultáneamente, el sistema permite la bidireccionalidad con la aplicación de Android (actuación de forma asíncrona) mediante una función de callback. Esta función permanece a la escucha de comandos entrantes desde la aplicación, cuando un usuario le da al botón de “enviar mensaje a pantalla” usará el endpoint sendMessage del servidor tomcat, que hará publish del mensaje escrito en la aplicación al mismo topic que está suscrito el ESP32 (Sensors/+/actuador/mensaje), usando wildcard para estar suscrito a todos los mensajes enviados por las distintas calles, dado a que contamos con una única pantalla real, si tuviésemos más el mensaje de cada calle iría a la pantalla que le corresponde, al estar suscrito al topic del que se envía el mensaje, recibe una alerta, por lo que interrumpe momentáneamente el ciclo de visualización para limpiar la pantalla LCD, proyectar el mensaje recibido y ejecutar un patrón de parpadeo en la retroiluminación (backlight blinking), asegurando así que la notificación capte la atención del usuario final.

Resultado Final del Prototipo

El resultado de este proceso de montaje y programación es un dispositivo compacto y totalmente funcional. Como se aprecia en la imagen inferior, la pantalla LCD actúa como el elemento central de la interfaz, mostrando en tiempo real los datos procesados y el estado de la conexión, mientras que los sensores periféricos capturan las condiciones del entorno para alimentar la simulación

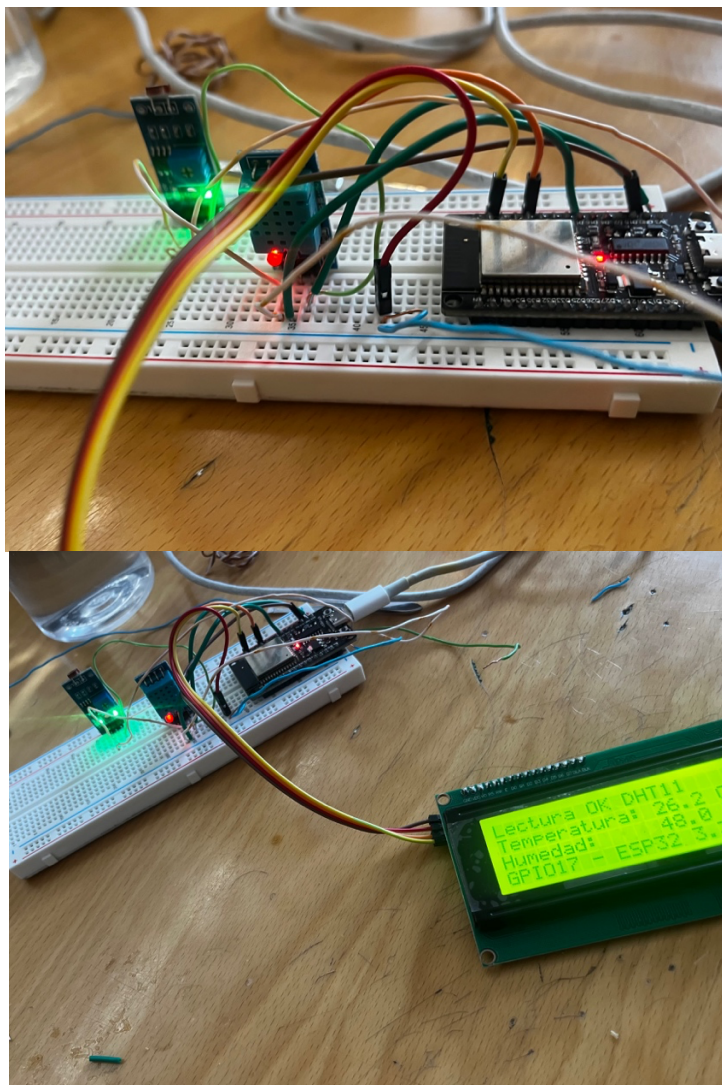


Ilustración 3 - Montaje real

4.5. Comunicación con otras capas

La integración efectiva del nodo IoT con las capas superiores es necesario para transformar un simple sensor aislado en un componente activo de una *Smart City*. Esta comunicación comienza con la conexión a la capa física de red, donde al iniciarse el firmware utiliza las librerías nativas del ESP32 para negociar el acceso al punto de acceso WiFi y obtener una dirección IP. Una vez garantizada la conectividad a Internet,

el dispositivo instancia un cliente PubSubClient que establece un túnel TCP persistente contra nuestro servidor Docker en el puerto 1883, esto actuará como puente digital entre el mundo físico y la lógica de negocio en la nube.

Para el flujo de información ascendente el sistema implementa un mecanismo de serialización de datos donde las lecturas crudas obtenidas de los sensores DHT11 y LDR son procesadas y estructuradas en cadenas de texto con formato JSON. Se creará un JSON para cada una de las distintas calles, y se enviarán a distintos topics dependiendo de la calle al que se refiera el JSON (`street_id`) permitiendo que las capas superiores distingan y clasifiquen los datos geográficamente de forma automática. A los topic de estos JSON enviados por el ESP32 están suscritos tanto el servidor de tomat, que procesará estos datos y los guardará en la base de datos de MariaDB, como la aplicación de Android para mostrar la información en tiempo real.

Por otro lado, el flujo descendente permite a la aplicación interactuar con los actuadores del hardware (pantalla). Esto lo logra usando el endpoint del API REST (`sendMessage`) que hará un publish con el mensaje que el usuario quiere que se imprima por pantalla, el ESP32 está suscrito a este topic y será notificado, entonces entra en la función callback que intercepta el mensaje, extrae el contenido de texto (`payload`) y toma el control del bus I2C para proyectar la alerta en la pantalla LCD, cerrando así el ciclo de comunicación bidireccional.

Finalmente, para garantizar la robustez de esta conexión frente a las posibles inestabilidades de una red inalámbrica real, se ha implementado una lógica de reconexión automática (Keep-Alive). A través de este se verifica el estado de la conexión MQTT y si falla entrará en modo recuperación hasta que funcione.

4.6. Pruebas realizadas

Para validar la robustez del prototipo y su integración con la arquitectura de la *Smart City*, se han ejecutado varias pruebas divididas en escenarios de funcionamiento nominal y escenarios de fallo controlado.

Casos de éxito (Funcionamiento nominal)

- **Validación de Percepción y Visualización:** Se comprobó que el ESP32 captura correctamente los valores del DHT11 y LDR. Estos se muestran periódicamente en la pantalla LCD 20x4 sin corrupción de caracteres, validando el correcto funcionamiento del bus I2C y la conversión de datos.
- **Persistencia de la Simulación Multi-calle:** Se verificó, accediendo a la base de datos a través de Adminer, que los mensajes simulados para las tres calles (Montes Albères, Pedro Gumiel y Paseo del Prado) se registran

correctamente en las tablas de MariaDB, respetando los tiempos y los identificadores de zona.

- **Obtención de datos en tiempo real:** Se verificó el correcto funcionamiento de la llegada de los datos en tiempo real de la aplicación Android, mediante el uso de MQTTExplorer para confirmar que los datos enviados por las el ESP32 de las distintas calles era el que mostraba la aplicación.
- **Mensaje de aplicación a pantalla:** Se confirmó la bidireccionalidad del sistema enviando mensajes de emergencia desde la App Android. El dispositivo recibió la orden mediante MQTT, interrumpió el ciclo de visualización y mostró el texto en el LCD activando el parpadeo de la retroiluminación en menos de 200ms.

Casos de error y manejo de excepciones

- **Pérdida de Conexión Wi-Fi:** Al apagar el punto de acceso, el firmware detectó la desconexión y activó la lógica de espera no bloqueante. El dispositivo reanudó el envío de datos automáticamente en cuanto la señal de red fue restablecida.
- **Bróker MQTT Inalcanzable:** Se simuló una caída del servidor deteniendo el contenedor Docker de Mosquitto. El ESP32 reportó el fallo en el puerto serie (código de retorno rc=-2) y ejecutó la función reconnect() en bucle hasta lograr restablecer el enlace TCP.
- **Gestión de Datos Nulos:** Se provocó un fallo físico desconectando el cable del sensor DHT11. El firmware detectó la lectura NaN y, en lugar de enviar datos corruptos a la nube, marcó el registro como inválido en el log serie para facilitar la depuración.

4.7. Capa de procesado

La Capa de Procesado es la encargada de dotar de inteligencia y persistencia al sistema. Su objetivo fundamental es orquestar el flujo de información bidireccional, transformando los datos crudos provenientes de la capa de percepción en información estructurada útil para la toma de decisiones, además de gestionar las peticiones de los usuarios finales a través de la aplicación móvil.

Para garantizar la escalabilidad y el desacoplamiento de servicios, esta capa se ha implementado sobre una infraestructura virtualizada que integra cuatro componentes: el Broker MQTT para la gestión de mensajería en tiempo real, el servidor de

aplicaciones Apache Tomcat donde reside la lógica de negocio y la API REST, la Base de Datos para el almacenamiento histórico de los registros de las distintas calles y el gestor de la base de datos Adminer para las comprobaciones del correcto funcionamiento y correcta obtención de los datos.

4.7.1. Base de datos

Para el sistema de almacenamiento persistente del proyecto, se ha seleccionado MariaDB, un sistema de gestión de bases de datos relacional (RDBMS) utilizado para garantizar la integridad y el histórico de la información. En nuestro despliegue, hemos priorizado la seguridad aislando el servicio en el puerto 3306, que no se expone al exterior y es accesible únicamente por el servidor Tomcat (mediante el pool de conexiones para meter los datos) y el gestor Adminer a través de la red interna Docker. Además, se ha configurado la persistencia mediante volúmenes para salvaguardar los datos y se utiliza un script de inicialización (init.sql) que automatiza la creación del esquema y usuarios al arrancar el contenedor.

El diseño del modelo de datos sigue un esquema relacional normalizado, cuyo objetivo principal es desacoplar la información, para ello hemos estructurado la base de datos en tres entidades fundamentales:

1. **CALLE:** Almacena los datos geográficos estáticos, y su clave primaria es `STREET_ID`, lo que garantiza que cada ubicación sea única en el sistema.
2. **DISPOSITIVO:** Registra cada unidad de hardware desplegada, para ello utiliza `SENSOR_ID` como identificador y se vincula a una calle específica mediante una clave foránea.
3. **TELEMETRIA:** Encargada de almacenar el histórico masivo de lecturas, que utiliza un ID autoincremental como clave primaria y guarda los valores junto con el mensaje original completo en formato JSON para tener un respaldo desnormalizado

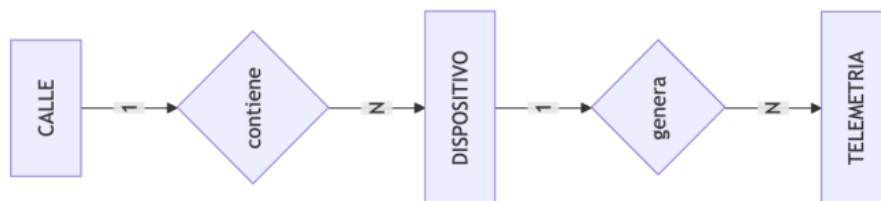


Ilustración 4 - Modelo relacional

Respecto a la cardinalidad y las relaciones entre estas entidades, el sistema se articula mediante relaciones de uno a muchos (1:N) que reflejan la realidad física del despliegue urbano:

- **Relación CALLE - DISPOSITIVO (1:N):** Una calle contiene múltiples dispositivos, pero un dispositivo físico solo puede estar instalado en una única calle. Esta relación se implementa propagando el STREET_ID de la tabla CALLE a la tabla DISPOSITIVO.
- **Relación DISPOSITIVO - TELEMETRÍA (1:N):** Un único dispositivo genera muchas lecturas de telemetría a lo largo del tiempo. Esta es la relación que soporta el crecimiento exponencial de los datos, vinculando cada registro de la tabla TELEMETRÍA con su sensor origen mediante el campo SENSOR_ID

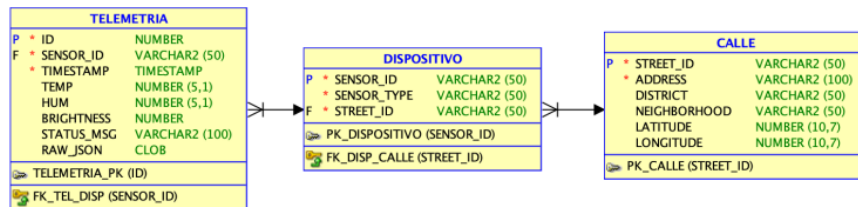


Ilustración 5 - Diagrama de tablas

4.7.2. Desarrollo del proyecto servidor

El proyecto se ha desarrollado sobre un servidor de aplicaciones Apache Tomcat 11, utilizando Java 17 como lenguaje de programación. Este entorno ha sido seleccionado por su robustez para gestionar aplicaciones web Java y su capacidad para mantener procesos en segundo plano necesarios para la escucha asíncrona de mensajes MQTT.

Para asegurar que en el proyecto las dependencias y entorno de ejecución sean idénticos en cualquier despliegue, hemos usado un Dockerfile Multi-stage:

1. **Etapas de Compilación (Maven):** Se utiliza una imagen base de Maven para compilar el código fuente y empaquetar la aplicación en un archivo .war. Esto gestiona automáticamente las dependencias del proyecto, como el conector de MySQL/MariaDB, la librería GSON para JSON y el cliente Paho MQTT.
2. **Etapas de Ejecución (Tomcat):** El .war generado se despliega sobre una imagen limpia de Tomcat 11 que expone el puerto 8080. A nivel de configuración interna, el servidor gestiona el acceso a la base de datos mediante un Pool de Conexiones (JNDI) definido en el fichero context.xml. Esto optimiza el rendimiento, permitiendo reutilizar las conexiones a MariaDB en lugar de abrir y cerrar una nueva por cada petición HTTP.

El proyecto sigue la estructura estándar de Apache Maven, separando el código fuente (src/main/java) de los recursos web (src/main/webapp). La aplicación se ha modularizado en paquetes según la funcionalidad de las clases

El paquete servlets constituye la interfaz API REST del servidor, conteniendo las clases que heredan de HttpServlet para actuar como controladores de la lógica web. Estos componentes interceptan las peticiones HTTP GET y POST dirigidas al contexto /ServerExampleUbicomp/ y procesan la respuesta para devolverla en formato JSON estructurado. Dentro de este contamos con endpoints como GetData y GetBySensor, diseñados para consultas históricas y filtrado específico, así como GetStats para ver las medias de las pantallas, GetByDate para ver los datos de los sensores de un día específico, SetData que permitirá inyectar datos vías HTTP y SendMessage, que gestiona la lógica de actuación hacia los dispositivos.

El paquete mqtt agrupa toda la lógica de mensajería asíncrona que se ejecuta en paralelo al servicio web. Cuenta con la clase ProjectInitializer, que implementa la interfaz ServletContextListener para ejecutarse automáticamente durante el arranque de Tomcat, lo que garantiza que el servidor se suscriba de inmediato al tópic (usando wildcard) Sensors/+sensor/telemetry para escuchar datos de cualquier calle sin intervención manual. Este paquete también incluye la clase utilitaria MQTTPublisher, encargada de inyectar en la red MQTT las órdenes de control generadas por los usuarios desde la aplicación móvil, para contar con bidireccionalidad.

Finalmente, el paquete model (database) implementa la capa de persistencia de datos siguiendo el patrón de diseño DAO. Este módulo contiene las clases responsables de mapear los objetos Java a las entidades relacionales de la base de datos, gestionando la conexión y la construcción eficiente de sentencias SQL. Es el encargado tanto de ejecutar las inserciones transaccionales de la telemetría entrante como de resolver las consultas complejas (joins y filtros) solicitadas por los servlets para generar las estadísticas. Las clases más importantes serán Logic que obtendrá los datos obtenidos del JSON, los separará y llamará a ConectionToDDBB para meterlos en la base de datos MariaDB mediante un pool de conexiones.

El servidor funciona como un sistema híbrido que combina dos patrones de comunicación, estos son ser un cliente pasivo y estar escuchando constantemente por si llegan datos nuevos y realizar peticiones a los servets cuando el usuario las necesite.

4.8. Capa de aplicación

La Capa de Aplicación constituye la interfaz tangible del sistema, y es la encargada de cerrar el ciclo de información entre la infraestructura técnica y el usuario final. Su objetivo fundamental es abstraer la complejidad para presentar los datos de las distintas calles de forma visual e intuitiva, además de habilitar la interacción bidireccional permitiendo el envío de mensajes hacia los dispositivos físicos desplegados en las calles.

Para garantizar tanto la accesibilidad como la capacidad de gestión, esta capa se ha desarrollado a través de dos plataformas complementarias: una Aplicación Móvil (Android) diseñada para la monitorización en tiempo real, acceso a registros de las calles y el control de actuadores por parte de los operarios, y una Aplicación Web orientada a la gestión y comprobamiento de la base de datos (MariaDB).

4.8.1. Aplicación móvil

La aplicación móvil del proyecto ha sido desarrollada para el sistema operativo Android, siendo esta aplicación un nodo activo de gestión urbana. Su arquitectura técnica combina dos protocolos de comunicación: HTTP (REST) mediante la librería Retrofit para las operaciones de gestión y consulta histórica, y MQTT para la monitorización en tiempo real. Esto permite que la aplicación centralice el control de los dispositivos desplegados (PECL1) y consuma eficientemente los servicios alojados en la infraestructura Docker (PECL2).

Flujo de Navegación y Pantallas (Activities)

1. Pantalla de Bienvenida (MainActivity) Al iniciar la aplicación, se presenta una pantalla de carga que muestra el logotipo de "AtmoCity" durante cuatro segundos. Esta actividad no requiere ningún tipo de interacción, ya que actúa como una transición automática que, tras finalizar el temporizador, lanza la siguiente pantalla y se cierra a sí misma para impedir el retorno de esta.

2. Selección de Calle (StreetSelection) Es el menú principal de navegación, para esta activity la aplicación utiliza Retrofit para realizar una petición GET al endpoint /GetCalles (creado para la PL3), obteniendo la lista actualizada de zonas disponibles para poblar un menú desplegable (*spinner*). Desde aquí, el usuario dispone de tres vías de acción:

- ☐ Seleccionar una calle y pulsar "Continuar" para ir al panel de monitorización.
- ☐ Acceder a "Añadir Calle" para registrar nuevos dispositivos.
- ☐ Entrar en "Ajustes" (SettingsActivity) para alternar el tema visual de la aplicación entre modo Claro y Oscuro.

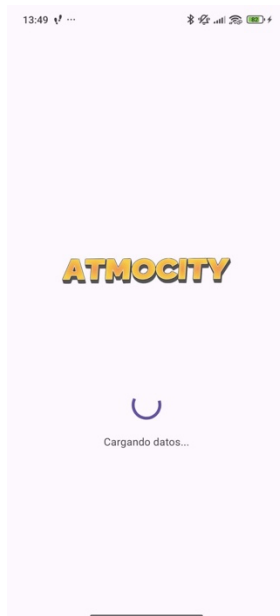
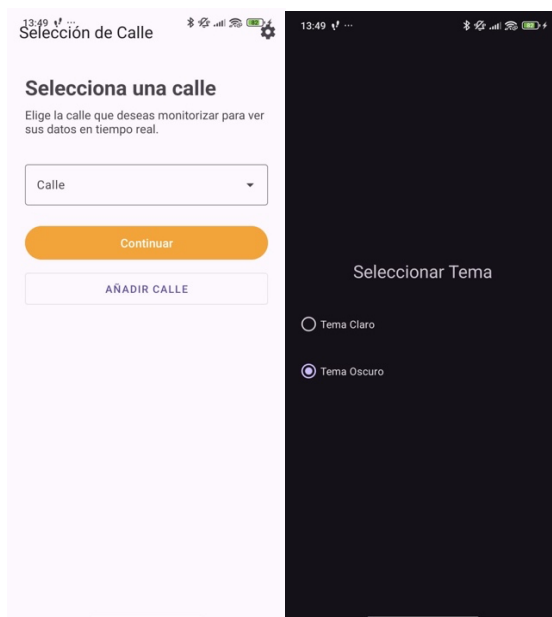


Ilustración 6 - Pantallas aplicación móvil



3. Inserción de Datos (AddDataActivity) Funciona como un formulario completo para el alta de un registro de una calle desde la aplicación, si la calle ya existe le añadirá un nuevo registro, mientras que si no está en la base de datos la crea. El usuario

introduce los identificadores del sensor, la calle, la ubicación (barrio, coordenadas) y los valores de medición. Antes del envío, la aplicación ejecuta validaciones lógicas en el cliente (ej. que la humedad esté entre 0-100% y la luminosidad sea 0 o 100). Si los datos son válidos, se empaquetan en un objeto DataPayload y se envían mediante una petición POST al endpoint /SetData.

13:49 Añadir Nuevo Registro Usado durante 25min

ID del Sensor (ej. SENSOR_01)

Tipo de Sensor (ej. test_device)

Datos de la Calle

ID de la Calle (ej. ST_CALLE_01)

Dirección (ej. Calle Falsa 123)

Distrito

Barrio

Latitud Longitud

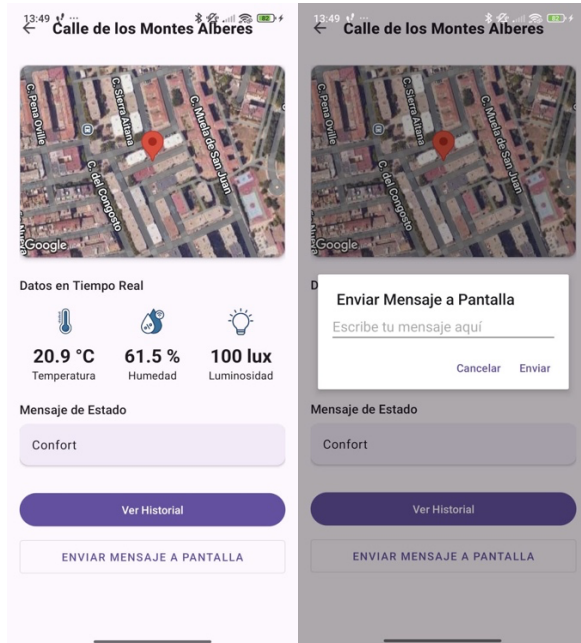
Valores de la Medición

Temperatura (ej. -5.5)

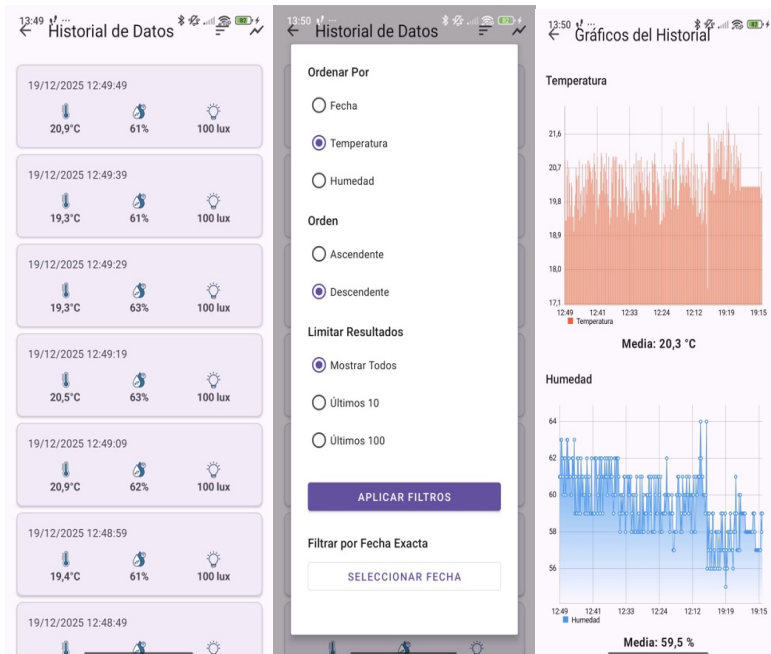
Humedad (ej. 60.2)

Luminosidad (0 o 100)

4. Monitorización en Tiempo Real de una calle (StreetMonitoring) Una vez hemos seleccionado una calle recibe el ID de la calle y se suscribe dinámicamente al tópico MQTT específico (Sensors/[Street_ID]/sensor/telemetry). Gracias a un MqttMessageListener, la interfaz se actualiza instantáneamente con cada nuevo dato recibido del ESP32, sin necesidad de refrescar la pantalla. Esta vista integra además la Google Maps API para geolocalizar el sensor (mediante la latitud y longitud) y habilita la bidireccionalidad: el botón "Enviar Mensaje a Pantalla" realiza una llamada GET al endpoint /SendMessage, enviando un texto que el servidor redirige al LCD del dispositivo físico.



5. Histórico de registros (HistoryActivity y GraphActivity) Si el usuario pulsa el botón de ver historial le lleva a la activity HistoryActivity, la cual consume el endpoint /GetData para recuperar todos los registros. Esta pantalla incorpora potentes filtros locales (ordenar por fecha, temperatura, limitar a los últimos 10/100 registros) y un filtrado por fecha específica que invoca al endpoint /GetByDate. Finalmente, la visualización avanzada se delega a la GraphActivity, que utiliza la librería MPAndroidChart para renderizar gráficos de líneas y barras, permitiendo comprender la evolución climática de la calle de un vistazo.



4.8.2. Aplicación Web

Para la comprobación del buen funcionamiento de la base de datos (MariaDB) se ha desplegado una aplicación web ligera basada en Adminer (servicio db-admin del docker). Esta herramienta actúa como la interfaz gráfica de usuario (GUI) para la gestión del sistema de almacenamiento, permitiendo a los administradores interactuar con la base de datos MariaDB directamente desde el navegador, sin necesidad de instalar software de escritorio pesado ni realizar configuraciones complejas de red.

La aplicación se ejecuta sobre un contenedor Docker independiente que expone el servicio en el puerto 8081, su implementación se llevó a cabo para garantizar seguridad explicada en la fase de infraestructura: dado que el contenedor de la base de datos no expone su puerto 3306 al exterior para mantenerse aislado y protegido, la aplicación web Adminer se despliega dentro de la misma red interna. Esto le permite conectarse localmente a la base de datos y actuar como una pasarela segura, ofreciendo un panel de control accesible externamente mientras mantiene la integridad del almacenamiento.

La interfaz de la aplicación web es minimalista y funcional, estructurándose en dos vistas principales que cubren todo el flujo de administración:

- **Pantalla de Autenticación:** Es el punto de acceso donde el administrador debe introducir las credenciales internas del sistema Docker, en él se debe especificar el servidor de base de datos por su nombre de host interno (mariadb-server), junto con el usuario (pecl2_user), la contraseña (1024) y el

nombre de la base de datos (UBICOMP). Esta pantalla valida los permisos y establece la conexión JDBC con el motor de datos.

- **Panel de Gestión:** Una vez autenticado, el usuario accede a la vista principal donde se listan las tablas de la base de datos: CALLE, DISPOSITIVO y TELEMETRIA. Desde esta interfaz la aplicación permite realizar operaciones como:
 - Visualización de Registros: Comprobar en tiempo real que los datos enviados por los sensores ESP32 (MQTT) se están persistiendo correctamente en las distintas tablas.
 - Ejecución SQL: Dispone de un área de Comando SQL para lanzar consultas complejas manuales, útiles para depuración o generación de informes rápidos.
 - Gestión del Esquema: Herramientas para importar/exportar la base de datos o modificar la estructura de las tablas si fuera necesario durante el desarrollo.

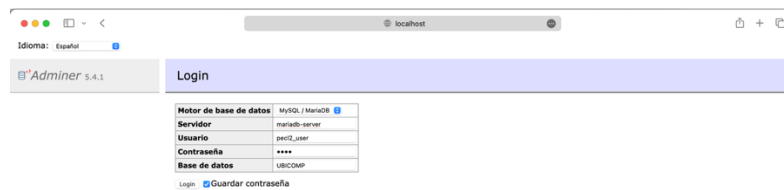
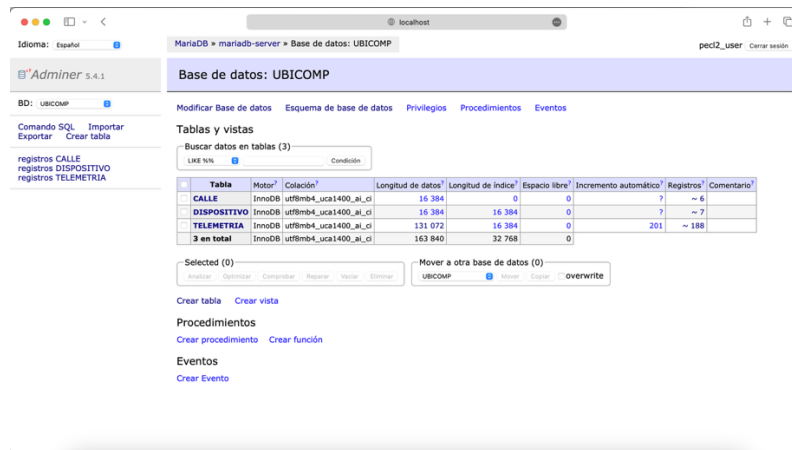


Ilustración 7 - Pantallas aplicación web



4.9. Planificación del proyecto

Para asegurar el éxito del proyecto se ha adoptado una metodología de trabajo ágil e incremental alineada con los principios de SCRUM. El ciclo de vida del desarrollo se ha estructurado en tres grandes iteraciones, coincidiendo con las fases de laboratorio de entrega de las PL, lo que ha permitido validar cada capa tecnológica antes de avanzar a la siguiente.

Metodología y Organización

- **Estrategia de Trabajo:** Se ha seguido un modelo colaborativo con reuniones de sincronización al inicio de cada sesión de trabajo para asignar objetivos a corto plazo. La gestión de tareas se ha visualizado mediante un tablero Kanban básico (Pendiente, En Progreso, Terminado).
- **Gestión del Código:** Se ha utilizado un repositorio de GitHub centralizado para el control de versiones, permitiendo la integración continua de los distintos lenguajes utilizados.
- **Equipo y Roles:** El equipo formado por Sergio Bardavio Soria y Ángel Jiménez Moneva, ha operado bajo un modelo de responsabilidad compartida (Full-Stack IoT). Ambos integrantes han participado activamente tanto en el montaje físico y soldadura como en la programación del servidor y la aplicación móvil, garantizando así un conocimiento global del sistema.

5. Conclusiones

Con la realización de este proyecto, hemos conseguido diseñar e implementar un sistema de computación ubicua completo y funcional, capaz de gestionar el ciclo de

vida de la información desde su captura en el entorno físico hasta su visualización en dispositivos móviles. Este trabajo ha supuesto un desafío de integramiento que ha unificado tecnologías heterogéneas, demostrando que la verdadera potencia del IoT no reside en los componentes aislados, sino en la capacidad de orquestar una comunicación fluida y robusta entre el hardware de campo, la infraestructura de servidor y las interfaces de usuario.

Respecto a la Capa de Percepción, se ha validado la viabilidad de utilizar microcontroladores de bajo coste como el ESP32 para aplicaciones de Smart City. La implementación del firmware ha ido más allá de la simple lectura de sensores (DHT11 y LDR), logrando simular un entorno complejo de múltiples calles mediante algoritmos de variación de datos. Esta lógica, combinada con la capacidad de actuación local a través de la pantalla LCD, ha dotado a los componentes de esta capa de una inteligencia suficiente para operar de forma autónoma y bidireccional, actuando no solo como emisor de datos sino también como punto de información ciudadana.

La robustez del sistema se ha desarrollado sobre la Capa de Procesado, donde la adopción de una arquitectura basada en contenedores Docker ha resultado fundamental. La separación de servicios ha garantizado un entorno escalable y seguro, donde la gestión de datos se ha optimizado mediante un modelo de base de datos relacional normalizado. Quedando así demostrada la eficacia de una arquitectura híbrida de comunicación, capaz de utilizar MQTT para la inmediatez de la monitorización en tiempo real y una API REST para la gestión estructural y el análisis histórico, asegurando así un uso eficiente del ancho de banda y los recursos del servidor.

La Capa de Aplicación es la encargada de transformar los datos técnicos en información accesible. El desarrollo de la aplicación móvil en Android ha permitido validar patrones de diseño complejos, como la gestión de hilos asíncronos para la red y la integración de librerías de terceros para la geolocalización y el análisis gráfico. La aplicación ha demostrado ser una herramienta de gestión potente, permitiendo al usuario no solo supervisar el estado ambiental de la ciudad desde cualquier lugar, sino también intervenir activamente sobre la infraestructura física mediante el envío de comandos remotos.

Finalmente, este proyecto ha servido para consolidar una visión Full-Stack del desarrollo en ingeniería telemática, mediante este se ha comprobado que la clave para construir soluciones tecnológicas viables reside en la interoperabilidad y la estandarización de protocolos. Desde el cableado físico hasta la inyección de dependencias en el servidor y la experiencia de usuario en el móvil, cada decisión de diseño ha impactado en el conjunto global, resultando en un prototipo final que cumple con los requisitos de escalabilidad, persistencia y tiempo real exigidos en los sistemas de monitorización modernos

6. Bibliografía

[1] Universidad de Alcalá, "Enunciado y plantilla PECL4: Documentación de un proyecto de computación ubicua," Feb. 2025.

[2] S. Bardavío Soria y A. Jimenez Moneva, "Memorias de prácticas PL1, PL2 y PL3" Universidad de Alcalá, 2025

[3] M. Braeuner, "Guía de Docker para principiantes: cómo crear tu primera aplicación Docker," *freeCodeCamp*, 18-Jul-2022. [En línea]. Disponible: <https://www.freecodecamp.org/espanol/news/guia-de-docker-para-principiantes-como-crear-tu-primera-aplicacion-docker/>.

[4] MariaDB Foundation, "MariaDB Knowledge Base: SQL Statements & Structure," 2024. [En línea]. Disponible: <https://mariadb.com/kb/en/documentation/>.

Anexo I – Manual de instalación

Este manual describe el procedimiento para desplegar el sistema AtmoCity desde cero en un entorno local.

Configuración de la Infraestructura (Backend)

1. **Archivo de Entorno:** Localice el archivo .env en la raíz de la carpeta ubicua-stack y verifique que contiene las siguientes credenciales:
 - REAL_DB_USER=pecl2_user
 - REAL_DB_PASS=1024
 - DB_ROOT_PASS=1024super
 - DB_NAME=pecl2_db
 - TOMCAT_PORT=8080
 - MQTT_PORT=1883

2. **Despliegue de Contenedores:** Abra una terminal en la ruta del proyecto y ejecute:

docker compose up -d --build

3. **Verificación de Base de Datos:** Acceda a *http://localhost:8081* (Adminer) y compruebe que el esquema UBICOMP y las tablas CALLE, DISPOSITIVO y TELEMETRIA se han creado mediante el script init.sql.

Configuración y Carga del Hardware (ESP32)

1. **Modificación del Firmware:** Abra el archivo .ino en el IDE de Arduino y localice la sección de configuración. Debe sustituir los valores por los de su entorno actual:

- **WiFi:**

```
const char* ssid = "POCO X7 Pro"; // Cambiar por el nombre de su red
const char* password = "Bardavio"; // Cambiar por su contraseña
```

- **Servidor MQTT:**

```
const char* mqtt_server = "192.168.0.28"; // Poner la IP local del PC donde corre Docker
```

2. **Carga de Código:** Conecte la placa ESP32, seleccione el puerto correspondiente y suba el programa. El monitor serie confirmará la conexión con el mensaje "WiFi conectado" e "Intentando conexión MQTT... conectado".

Despliegue de la Aplicación Android

1. **Configuración de IP en el código:** En el proyecto de Android Studio, busque la clase donde se define la conexión a la API (dentro de la configuración de Retrofit y del Archivo StreetMonitoring) y cambie la URL base para que apunte a la IP de su servidor:

- *BASE_URL = "http://192.168.0.28:8080/ServerExampleUbicomp/"*

2. **Ejecución:**

- Active la Depuración USB en su teléfono móvil.
- Conecte el terminal al PC y pulse el botón Run en Android Studio.
- La aplicación se instalará y permitirá seleccionar entre las calles configuradas para ver sus datos en tiempo real e historial de registros.

Anexo II – Manual de usuario de la aplicación móvil

1. Introducción

Este manual de usuario te guiará a través de todas las funcionalidades de la aplicación de monitorización de calles. Con esta aplicación, podrás consultar datos ambientales en tiempo real, visualizar la ubicación de las calles, analizar el historial de mediciones, añadir nuevas vías al sistema y enviar mensajes a paneles informativos.

2. Pantalla de Inicio

Al ejecutar la aplicación, una pantalla de bienvenida con el logo y una barra de progreso se mostrará durante unos segundos. Pasado este tiempo, serás redirigido automáticamente a la pantalla principal.

3. Pantalla de Selección de Calles

Esta es la pantalla principal de la aplicación. Aquí podrás:

- **Seleccionar una calle:** Toca el menú desplegable para ver una lista de todas las calles disponibles. Selecciona una y pulsa el botón "Continuar" para ver sus datos.
 - **Añadir una nueva calle:** Pulsa el botón "Añadir Calle". Se abrirá un nuevo formulario donde deberás introducir todos los datos necesarios para registrar una nueva calle.
 - **Actualizar la lista:** Desliza el dedo hacia abajo sobre la pantalla. La aplicación se volverá a conectar con el servidor para obtener la lista de calles más reciente.
 - **Acceder a Ajustes:** En la esquina superior derecha, encontrarás un menú que te permitirá acceder a la pantalla de Ajustes, desde donde podrás, por ejemplo, cambiar el tema de la aplicación (claro/oscuro).

4. Pantalla de Monitorización de Calle

Una vez seleccionada una calle, accederás a la pantalla de monitorización, que se divide en varias secciones:

- **Datos en Tiempo Real:** En la parte superior, verás los valores actuales de temperatura, humedad y luminosidad, que se actualizan automáticamente. También se muestra un mensaje de estado.
- **Mapa de Ubicación:** Se mostrará un mapa satélite con un marcador que indica la ubicación geográfica exacta de la calle seleccionada.
- **Enviar Mensaje a Pantalla:** Al pulsar el botón "Enviar Mensaje", aparecerá una ventana emergente. Escribe el texto que desees mostrar en la pantalla inteligente de la calle y pulsa "Enviar".
- **Ver Historial:** Al pulsar el botón "Ver Historial", serás dirigido a la pantalla con el registro histórico de datos de esa calle.

5. Pantalla de Historial de Datos

En esta sección, podrás analizar en detalle las mediciones pasadas de la calle.

- **Lista de Registros:** Verás una lista con todas las mediciones históricas, mostrando la fecha, hora y los valores de los sensores en ese momento.
- **Menú de Opciones** (esquina superior derecha):

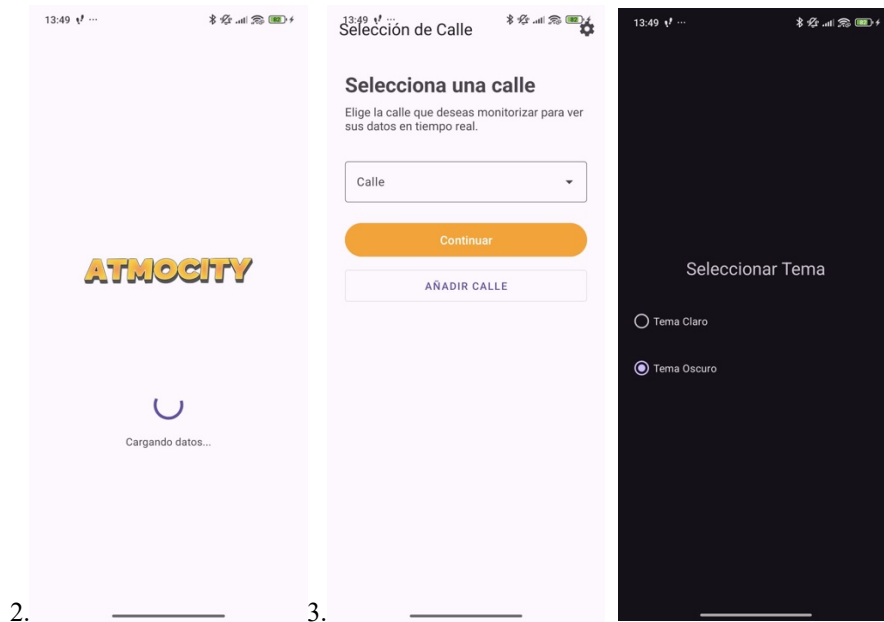
◦Filtrar y Ordenar: Te permite organizar y acotar los datos mostrados. Al seleccionarlo, se abre un diálogo con las siguientes opciones:

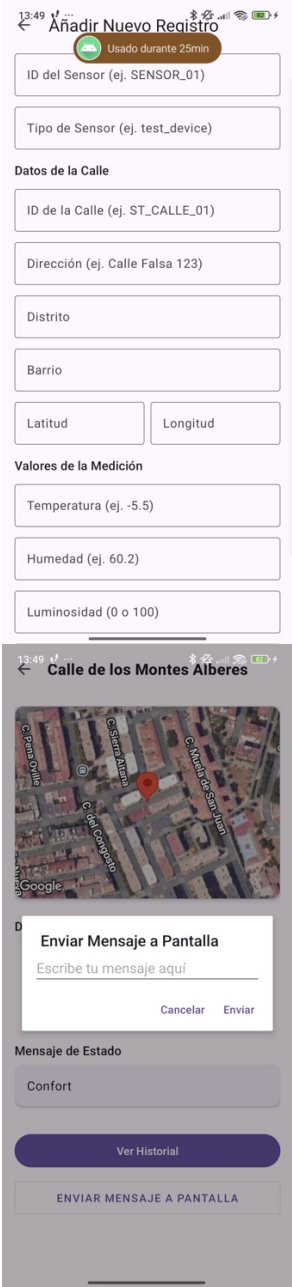
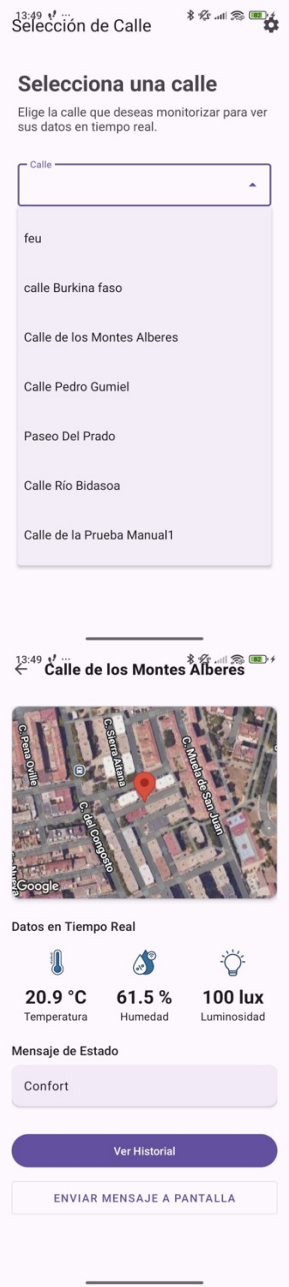
- Ordenar por: Fecha, Temperatura o Humedad.
- Orden: Ascendente o Descendente.
- Limitar a: 10, 100 o todos los resultados.
- Seleccionar Fecha: Abre un calendario para que puedas ver los registros de un día concreto.
- Resetear Filtros: Elimina todos los filtros aplicados y vuelve a mostrar la lista completa.

◦Ver Gráfico: Muestra los datos históricos en un gráfico interactivo, permitiéndote visualizar las tendencias a lo largo del tiempo.

6. Solución de Problemas Comunes

- "Error de conexión. Revisa la IP...": Este mensaje aparece si la aplicación no puede conectar con el servidor. Asegúrate de que tu dispositivo móvil está en la misma red WiFi que el servidor.
- Los datos no se actualizan: Intenta refrescar la lista de calles (deslizando hacia abajo) o vuelve a la pantalla anterior y selecciona la calle de nuevo.
- No aparece ninguna calle en la lista: Asegúrate de que el servidor está en funcionamiento y que hay calles registradas en la base de datos. Puedes usar el gesto de "deslizar para refrescar".





4.



5.

13:50 Historial de Datos

Ordenar Por

- ☐ Fecha
- ☒ Temperatura
- ☐ Humedad

Orden

- ☐ Ascendente
- ☒ Descendente

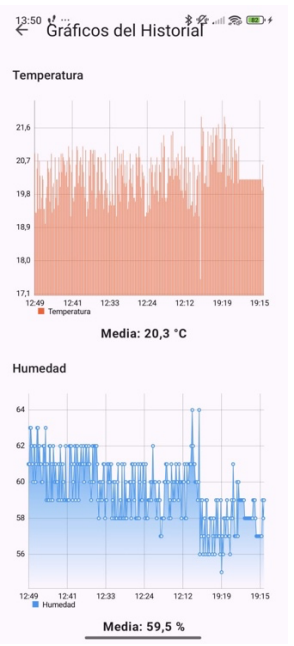
Limitar Resultados

- ☒ Mostrar Todos
- ☐ Últimos 10
- ☐ Últimos 100

APLICAR FILTROS

Filtrar por Fecha Exacta

SELECCIONAR FECHA



Anexo III – Manual de usuario de la aplicación web

El sistema AtmoCity incluye una interfaz web de gestión basada en Adminer. Esta herramienta permite a los administradores supervisar en tiempo real la entrada de datos de los sensores, gestionar el inventario de calles y dispositivos, y ejecutar consultas directamente sobre el motor MariaDB sin necesidad de software adicional.

1. Acceso al Sistema

Para acceder a la aplicación, el usuario debe abrir un navegador web en la misma red donde se encuentra desplegado el servidor y navegar a la dirección: *http://localhost:8081*.

Formulario de Login: Aparecerá una pantalla de autenticación donde se deben introducir los credenciales configurados en el archivo .env:

- ☐ **Motor de base de datos:** MySQL / MariaDB.
- ☐ **Servidor:** mariadb-server.
- ☐ **Usuario:** pecl2_user.
- ☐ **Contraseña:** 1024.
- ☐ **Base de datos:** UBICOMP.

Tras pulsar el botón Login, se accederá al panel de control principal.

2. Gestión de Tablas y Esquema

Una vez dentro, el usuario visualizará el esquema de la base de datos UBICOMP. En el menú lateral izquierdo y en el panel central se muestran las tres tablas que componen el sistema:

- ☐ **CALLE:** Almacena los datos geográficos estáticos y direcciones.
- ☐ **DISPOSITIVO:** Inventario de hardware que vincula cada sensor a una ubicación.
- ☐ **TELEMETRIA:** Histórico masivo de lecturas ambientales.

El administrador dispone de herramientas de mantenimiento global para:

- ☐ Modificar Base de datos: Alterar parámetros del esquema.
- ☐ Esquema de base de datos: Visualización gráfica de las relaciones entre tablas.
- ☐ Privilegios, Procedimientos y Eventos: Gestión avanzada de usuarios, funciones y tareas programadas en la base de datos.
- ☐ Importar/Exportar: Permite realizar copias de seguridad o cargar datos masivos.
- ☐ Crear tabla/vista: Expansión de la estructura del sistema.

3. Visualización y Monitorización de Datos

El uso principal para el administrador es la supervisión de las lecturas enviadas por el ESP32:

- ☐ Consulta de las tablas: Al seleccionar una tabla se muestra un listado con las columnas de esta, para telemetría serían estas: ID, SENSOR_ID, TIMESTAMP, TEMP, HUM, BRIGHTNESS, STATUS_MSG y el RAW_JSON completo.

- **Búsqueda y Filtrado Avanzado:** Adminer permite localizar registros específicos utilizando operadores lógicos en cualquier columna. Los operadores disponibles incluyen: =, <, >, <=, >=, !=, LIKE, LIKE %% (búsqueda parcial), REGEXP, IN, IS NULL, entre otros. Esto es ideal para encontrar, por ejemplo, todos los registros de un sensor específico en un rango de temperatura concreto.
- **Análisis de Inventario:** En la tabla CALLE, el usuario puede revisar las coordenadas (LATITUDE, LONGITUDE) y direcciones de las ubicaciones simuladas, como la Calle de los Montes Albères o Paseo del Prado.
- **Estadísticas de Almacenamiento:** El panel principal muestra métricas de salud de las tablas como el número de registros (ej. ~2500 en Telemetría), longitud de datos, espacio libre e incremento automático de los índices.

4. Consultas SQL Personalizadas

Para usuarios avanzados, Adminer permite ejecutar comandos SQL directos para un control total:

1. Haga clic en "Comando SQL" en el menú lateral.
2. Escriba la sentencia en el cuadro de texto para realizar operaciones complejas de análisis o limpieza de datos (ej: `SELECT * FROM TELEMETRIA WHERE TEMP > 25`).
3. Pulse "Ejecutar" para obtener resultados inmediatos

Idioma: Español

Adminer 5.4.1

Login

Motor de base de datos: MySQL / MariaDB

Servidor

Usuario

Contraseña

Base de datos

mariadb-server

pec22_user

UBICOMP

Log in Guardar contraseña

Idioma: Español

Adminer 5.4.1

MariaDB » mariadb-server » Base de datos: UBICOMP

pec22_user Cerrar sesión

Base de datos: UBICOMP

Modificar Base de datos

Esquema de base de datos

Privilegios

Procedimientos

Eventos

Tablas y vistas

Buscar datos en tablas (3)

LIKE %N% Condiciones

#	Tabla	Motor	Colación	Longitud de datos	Longitud de índice	Espacio libre	Incremento automático	Registros	Comentario
1	CALLE	InnoDB	utf8mb4_uca1400_ai_ci	16 384	0	0	?	~ 6	
2	DISPOSITIVO	InnoDB	utf8mb4_uca1400_ai_ci	16 384	16 384	0	?	~ 7	
3	TELEMETRIA	InnoDB	utf8mb4_uca1400_ai_ci	131 072	16 384	0		201 ~ 188	
3 en total		InnoDB	utf8mb4_uca1400_ai_ci	163 840	32 768	0			

Selected (0)

Analizar Optimizar Comparar Reparar Vaciar Eliminar

Mover a otra base de datos (0)

UBICOMP Guardar Local Overwrite

Crear tabla Crear vista

Procedimientos

Crear procedimiento Crear función

Eventos

Crear Evento

BD: UBICOMP

Comando SQL Importar Exportar Crear tabla

registros CALLE

registros DISPOSITIVO

registros TELEMETRIA

Comando SQL

SELECT

INSERT INTO

UPDATE

DELETE FROM

EXPLAIN

Ejecutar Limit rows: Parar en caso de error Mostrar solamente errores

Idioma: Español

MariaDB » mariadb-server » UBICOMP » Mostrar: DISPOSITIVO

pec22_user Cerrar sesión

Mostrar: DISPOSITIVO

Visualizar contenido

Mostrar estructura

Modificar tabla

Nuevo Registro

Mostrar

Condición

Ordenar

Limite

Longitud de texto

Acción

50

100

Mostrar

SELECT * FROM 'DISPOSITIVO' LIMIT 58 (0.002 s) [Modificar](#)

	SENSOR_ID	SENSOR_TYPE	STREET_ID
Modificar	ID_01	sensor_pantalla	ID_01
Modificar	ID_02	pantalla	ST_02
Modificar	SENSOR_POSTMAK_R2	postman_client	Calle Mayor 12, Madrid
Modificar	SENSOR_PRUEBA_MANUAL	test_device	ST_PRUEBA_MANUAL
Modificar	ST_0155_ESP32_01	information_display	ST_0155
Modificar	ST_0156	information_display	ST_0155
Modificar	ST_0156_ESP32_01	information_display	ST_0156
Modificar	ST_0157_ESP32_01	pantalla	ST_0157

Resultado completo

8 registros

Modificar

Selected (0)

Exportar (8)

[Importar](#)

Idioma: Español

MariaDB » mariadb-server » UBICOMP » Mostrar: TELEMETRIA

pec22_user Cerrar sesión

Mostrar: TELEMETRIA

Visualizar contenido

Mostrar estructura

Modificar tabla

Nuevo Registro

Mostrar

Condición

Ordenar

Limite

Longitud de texto

Acción

50

100

Mostrar

SELECT * FROM 'TELEMETRIA' LIMIT 58 (0.002 s) [Modificar](#)

	ID	SENSOR_ID	TIMESTAMP	TEMP	HUM	BRIGHTNESS	STATUS_MSG
Modificar	1	ST_0156	2025-12-16 19:15:16	20.2	57	0	Temp. Comfort
Modificar	2	ST_0156	2025-12-16 19:15:21	20.2	57	0	Temp. Comfort
Modificar	3	ST_0156	2025-12-16 19:15:26	20.2	57	0	Temp. Comfort
Modificar	4	ST_0156	2025-12-16 19:15:31	20.2	57	0	Temp. Comfort
Modificar	5	ST_0156	2025-12-16 19:15:36	20.2	57	0	Temp. Comfort
Modificar	6	ST_0156	2025-12-16 19:16:47	20.2	57	0	Temp. Comfort
Modificar	7	ST_0156	2025-12-16 19:16:52	20.2	57	0	Temp. Comfort
Modificar	8	ST_0156	2025-12-16 19:17:33	20.2	57	0	Temp. Comfort
Modificar	9	ST_0156	2025-12-16 19:17:38	20.2	58	0	Temp. Comfort
Modificar	10	ST_0156	2025-12-16 19:17:43	20.2	59	0	Temp. Comfort
Modificar	11	ST_0156	2025-12-16 19:17:48	20.2	59	0	Temp. Comfort
Modificar	12	ST_0156	2025-12-16 19:17:53	20.2	59	0	Temp. Comfort
Modificar	13	ST_0156	2025-12-16 19:17:58	20.2	58	0	Temp. Comfort
Modificar	14	ST_0156	2025-12-16 19:18:03	20.2	58	0	Temp. Comfort
Modificar	15	ST_0156	2025-12-16 19:18:08	20.2	58	0	Temp. Comfort
Modificar	16	ST_0156	2025-12-16 19:18:13	20.2	58	0	Temp. Comfort
Modificar	17	ST_0156	2025-12-16 19:18:18	20.2	58	0	Temp. Comfort

Página

1 2 3 4

Resultado completo

200 registros

Modificar

Selected (0)

Exportar (200)

Anexo IV – Hojas de características

El presente anexo recoge las especificaciones técnicas de los principales componentes hardware empleados en el desarrollo del sistema AtmoCity. La correcta selección e integración de estos dispositivos, atendiendo a sus rangos de funcionamiento y a los protocolos de comunicación utilizados, resulta fundamental para garantizar la fiabilidad de las mediciones ambientales y la estabilidad operativa del nodo sensor.

1. Microcontrolador: DOIT ESP32 DevKit V1

El microcontrolador DOIT ESP32 DevKit V1 constituye la unidad central de procesamiento del sistema, siendo responsable de la adquisición de datos procedentes de los sensores, la ejecución de la lógica de control y la gestión de las comunicaciones inalámbricas.

Alimentación y voltaje de operación: La lógica interna opera a una tensión de 3,3 V. El módulo permite alimentación externa a través del pin VIN.

Unidad de procesamiento: Integra una CPU Tensilica Xtensa LX6 de 32 bits, configurable en arquitectura de uno o dos núcleos, con una frecuencia de reloj máxima de 240 MHz.

Memoria: Dispone de 4 MB de memoria Flash para almacenamiento de programa y 520 KB de memoria SRAM para ejecución de procesos.

Capacidades de comunicación: Incluye conectividad inalámbrica Wi-Fi y Bluetooth integradas, adecuadas para aplicaciones IoT.

Asignación de pines en el sistema:

- ☐ GPIO 17: Entrada de datos del sensor de humedad y temperatura DHT11.
- ☐ GPIO 25: Entrada analógica procedente del sensor de iluminación.
- ☐ GPIO 21 (SDA) y GPIO 22 (SCL): Bus I2C destinado a la comunicación con la pantalla LCD.

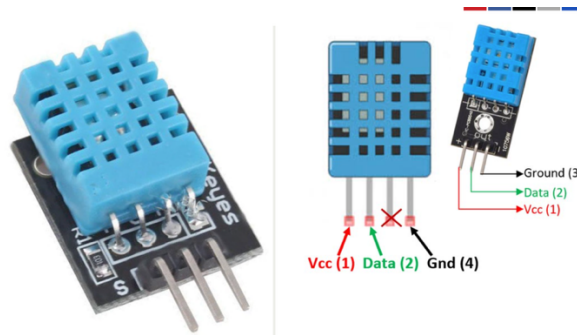


Ilustración 9 - Sensor de humedad y temperatura DHT11

https://www.mouser.com/datasheet/2/758/DHT11-Technical-Data-Sheet-Translated-Version-1143054.pdf?srltid=AfmBOorh9AiSmsigdrwZS_AfdHBGUSE7GTiMs73-tdxJh1a27ZkpdMcs

3. Sensor de iluminación: GL5528

El módulo GL5528 permite la estimación de la intensidad luminosa ambiental mediante el uso de una fotorresistencia del tipo LDR, cuya resistencia varía en función de la luz incidente.

Alimentación:

Compatible con niveles de tensión de 3,3 V y 5 V.

Principio de funcionamiento: La resistencia eléctrica del sensor disminuye conforme aumenta la intensidad luminosa, permitiendo la conversión de la señal óptica en una señal eléctrica.

Tipos de salida: El módulo ofrece salida digital, porque su salida se limita a dos estados discretos.

Ajuste del umbral: Incorpora un potenciómetro para la configuración del nivel de disparo de la salida digital.

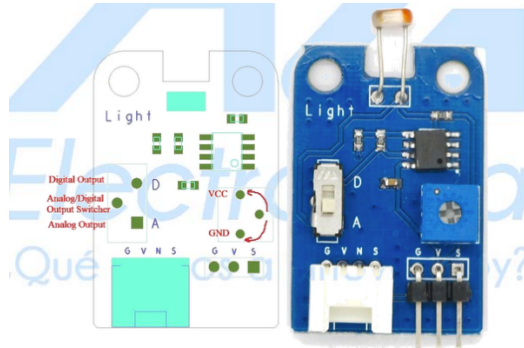


Ilustración 10 - Sensor de iluminación GL5528

<https://agelectronica.lat/pdfs/textos/I/IM120710.PDF>

4. Pantalla LCD 20x4 con interfaz I2C

La pantalla LCD 20x4 se emplea como elemento de visualización local del sistema, permitiendo la presentación de información textual de forma clara y estructurada.

Características del display: Pantalla alfanumérica con cuatro líneas de veinte caracteres cada una.

Controlador: Basada en un controlador compatible con el estándar HD44780.

Interfaz de comunicación: Integra un módulo adaptador que permite la comunicación mediante el bus I2C, reduciendo el número de pines necesarios en el microcontrolador.

Tensión de alimentación: Funciona con una alimentación de 5 V.

Funciones adicionales: Dispone de ajuste de contraste mediante potenciómetro integrado y control de la retroiluminación mediante software.



Ilustración 11- Pantalla LCD 20x4 con interfaz I2C

https://www.handsontec.com/dataspecs/I2C_2004_LCD.pdf